

IMPLEMENTATION DOCUMENTATION

Finance Dashboard API

Backend Engineering — Assignment Submission

FastAPI · SQLAlchemy 2.0 · SQLite · Alembic · JWT RBAC

Project	Finance Dashboard Backend System
Stack	Python 3.12 · FastAPI · SQLAlchemy 2.0 (async) · SQLite · Alembic · python-jose · passlib · aiosqlite
Architecture	Layered: Endpoint → Service → Repository → Database
Auth	JWT access + refresh tokens · RBAC via require_permission() dependency factory · Bcrypt

1. System Overview

This document describes the backend implementation of a Finance Dashboard API built to production-grade engineering standards. The system manages financial records, enforces role-based access control, and exposes a clean versioned REST API designed to serve a frontend dashboard.

1.1 Core Requirements Addressed

Requirement	Status	Implementation
User & Role Management	Complete	Users, Roles, Permissions with full RBAC — 3 default roles seeded on startup
Financial Records CRUD	Complete	Create / Read / Update / Void with category type validation and audit trail

Dashboard Summary APIs	Complete	5 aggregation endpoints — summary, by-category, trends, activity, CSV export
Access Control Logic	Complete	require_permission() factory dependency declared per route — self-documenting
Validation & Error Handling	Complete	Pydantic v2 field validators + typed custom exceptions + registered error handlers
Data Persistence	Complete	SQLite via async SQLAlchemy 2.0 + aiosqlite + Alembic migrations

2. Database — SQLite

2.1 Why SQLite

SQLite was selected as the persistence layer for this assignment for the following deliberate reasons:

- Zero infrastructure overhead — no separate database server process, no Docker dependency, and no connection credentials required for a reviewer to run the project
- File-based persistence — a single finance.db file contains the entire database state, making the submission completely self-contained and reproducible
- Full SQL compliance — all queries, constraints, migrations, and aggregation logic (GROUP BY, CASE WHEN, SUM, COUNT) used in this project run identically on SQLite and PostgreSQL
- Async-compatible — accessed through SQLAlchemy's async engine with aiosqlite, maintaining the same async/await pattern that would be used with PostgreSQL in production

**NOT
E**

Production migration path: switching to PostgreSQL requires changing only DATABASE_URL in .env from sqlite+aiosqlite:///./finance.db to postgresql+asyncpg://user:pass@host/db. No application code changes are needed — the entire codebase is database-agnostic.

2.2 Async Engine Configuration

The async engine is configured in app/db/session.py:

```
engine = create_async_engine(
```

```

settings.DATABASE_URL,          # sqlite+aiosqlite:///./finance.db
echo=settings.DB_ECHO,          # logs SQL in development when True
connect_args={
    "check_same_thread": False, # required for SQLite with async access
},
)

AsyncSessionLocal = async_sessionmaker(
    bind=engine,
    class_=AsyncSession,
    expire_on_commit=False,      # prevents lazy-load errors after commit
    autoflush=False,
    autocommit=False,
)

```

2.3 Normalised Schema — Eight Tables

Table	Type	Purpose
users	Entity	User accounts — identity, credentials, active status, timestamps
roles	Entity	Named roles: viewer, analyst, admin
permissions	Entity	Fine-grained resource:action capability pairs (17 defined)
user_roles	Association	Many-to-many join — which user holds which role (plain Table)
role_permissions	Association	Many-to-many join — which role grants which permission (plain Table)
categories	Entity	Income / expense category master data with system flag
financial_records	Entity	Core financial entries — amount, type, status, category, date
audit_logs	Append-only	Immutable history of every mutating operation with before/after JSON

2.4 Key Schema Decisions

Soft deletes everywhere

Users are deactivated via `is_active = false`. Records are voided via `status = 'void'`. Neither is ever hard-deleted. This preserves referential integrity for audit logs and ensures historical aggregation totals remain accurate.

DB-level CHECK constraints

Critical enum fields are enforced at the database layer, not just the application layer:

```
__table_args__ = (
    CheckConstraint("type IN ('income', 'expense')", name='ck_record_type'),
    CheckConstraint("status IN ('draft', 'posted', 'void')", name='ck_record_status'),
    CheckConstraint('amount > 0',
name='ck_record_amount_positive'),
)
```

Plain Table objects for pure join tables

`user_roles` and `role_permissions` carry no extra columns so they are declared as SQLAlchemy Table objects, not mapped classes. Using `secondary=` on relationships with a mapped association class causes a `back_populates` conflict that is impossible to resolve — the plain Table pattern eliminates this entirely.

Audit log excludes TimestampMixin

`AuditLog` deliberately omits `updated_at` because audit rows must never be modified after creation. The `actor_id` FK uses `ON DELETE SET NULL` so the audit history is preserved if a user account is later removed.

3. Application Lifespan — Startup & Shutdown

3.1 Why Lifespan over `@app.on_event`

FastAPI's `@app.on_event('startup')` decorator is deprecated as of FastAPI 0.93. The modern pattern is the `asynccontextmanager` lifespan, which co-locates startup and shutdown logic in a single function and makes the resource lifecycle explicit and testable:

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    # everything before yield runs at STARTUP
    await run_startup()
    yield
    # everything after yield runs at SHUTDOWN
```

```

    await run_shutdown()

app = FastAPI(lifespan=lifespan)

```

3.2 What Runs at Startup

Step	Operation	Detail
1	Logging setup	structlog is configured first so every subsequent log call during startup is correctly formatted from the very first line
2	Table creation	SQLAlchemy create_all() runs against the async engine. Creates finance.db and all tables if they do not exist. Fully idempotent — existing tables are never dropped or altered.
3	Role & permission seed	seed_roles_and_permissions() inserts the 17 permission rows and 3 role rows with their assignments. Fully idempotent — existing rows are detected and skipped, never duplicated.

3.3 Complete Lifespan Implementation

```

# main.py
from contextlib import asynccontextmanager
from fastapi import FastAPI
from app.api.v1.routers import v1_router
from app.api.error_handlers import register_exception_handlers
from app.core.logging_config import setup_logging
from app.db.session import engine, AsyncSessionLocal
from app.db.seed import seed_roles_and_permissions
from app.models import Base  # registers all models with Base.metadata

@asynccontextmanager
async def lifespan(app: FastAPI):
    # — STARTUP —
    setup_logging()  # Step 1: logging first

    async with engine.begin() as conn:  # Step 2: create all tables
        await conn.run_sync(Base.metadata.create_all)

    async with AsyncSessionLocal() as db:  # Step 3: seed master data
        await seed_roles_and_permissions(db)

    yield  # application is now live

    # — SHUTDOWN —
    await engine.dispose()  # close all DB connections

```

```
def create_app() -> FastAPI:
    app = FastAPI(title="Finance Dashboard API", version="1.0.0",
                  docs_url="/api/docs", lifespan=lifespan)
    register_exception_handlers(app)
    app.include_router(v1_router)
    return app

app = create_app()
```

3.4 Why Seeding Runs in Lifespan, Not Alembic

Alembic migration seeding	Lifespan seeding (our approach)
Runs once only — permission changes need a new migration just for data	Runs on every startup — always in sync with code changes automatically
Reviewer must run alembic upgrade head before starting the app	Reviewer runs uvicorn main:app — DB, tables, and seed data ready immediately
Data migrations mixed into the schema migration history	Clean separation: Alembic owns schema shape, lifespan owns master reference data

4. Architecture

4.1 Layered Architecture

Every HTTP request passes through four layers in strict order. Each layer has a single responsibility and depends only on the layer directly below it.

Layer	Location	Responsibility
Endpoint	<code>app/api/v1/endpoints/</code>	Declares HTTP method, path, query params, response schema. Delegates immediately to service. Contains zero business logic.
Service	<code>app/services/</code>	Business rules, validation, orchestration. Calls repository for data. Calls <code>write_audit()</code> after every mutation.

Repository	<code>app/repositories/</code>	All SQLAlchemy queries. Pagination, filtering, ordering. Raises domain exceptions only — never HTTPException.
Database	<code>app/models/ + app/db/</code>	ORM models, Base.metadata, async engine, session factory, Alembic migrations.

4.2 RBAC Implementation

The permission guard is a factory function that returns a FastAPI dependency with the required resource and action baked in. This makes permissions self-documenting at the route declaration:

```
def require_permission(resource: str, action: str):
    async def _check(current_user: User = Depends(get_current_user)) -> User:
        for role in current_user.roles:
            for perm in role.permissions:
                if perm.resource == resource and perm.action == action:
                    return current_user
            raise HTTPException(status_code=403,
                                detail=f'Permission denied: {action} on {resource}')
        return _check

# Route declaration is self-documenting:
@router.post("", response_model=RecordResponse, status_code=201)
async def create_record(
    payload: RecordCreate,
    current_user: User = Depends(require_permission("financial_records", "create")),
    db: AsyncSession = Depends(get_db),
):
    return await RecordService(db).create_record(payload, actor_id=current_user.id)
```

4.3 Permission Matrix

Permission	Viewer	Analyst	Admin
financial_records : read	✓	✓	✓
financial_records : create	—	—	✓
financial_records : update	—	—	✓
financial_records : delete (void)	—	—	✓
financial_records : read_history	—	✓	✓
analytics : read	✓	✓	✓

analytics : read_trends	—	✓	✓
analytics : export	—	✓	✓
categories : create / delete	—	—	✓
users : list / create / update	—	—	✓
roles : manage	—	—	✓

5. API Contract Summary

All endpoints are versioned under `/api/v1`. Interactive Swagger documentation is available at `/api/docs` once the server is running. Authentication uses Bearer token in the Authorization header.

Auth

Method	Path	Description	Access
POST	<code>/auth/login</code>	Exchange email + password for JWT access and refresh tokens	public
POST	<code>/auth/refresh</code>	Issue new access token from a valid refresh token	public

Users

Method	Path	Description	Access
GET	<code>/users/me</code>	Authenticated user's own profile and roles	all roles
GET	<code>/users</code>	Paginated + filterable user list	admin
POST	<code>/users</code>	Create user with role assignment	admin
PATCH	<code>/users/{user_id}</code>	Update profile, status, or roles	admin
DELETE	<code>/users/{user_id}</code>	Soft-deactivate (<code>is_active = false</code>)	admin

Financial Records

Method	Path	Description	Access
GET	/records	Paginated + filtered record list	viewer+
GET	/records/{id}	Single record with full relations	viewer+
POST	/records	Create record — validates category type match	admin
PATCH	/records/{id}	Partial update — blocked on void records	admin
DELETE	/records/{id}	Void record (status = void)	admin
GET	/records/{id}/history	Full audit trail for a record	analyst+

Dashboard & Analytics

Method	Path	Description	Access
GET	/dashboard/summary	Total income, expenses, net balance KPIs	viewer+
GET	/dashboard/by-category	Totals grouped by category with percentages	viewer+
GET	/dashboard/trends	Monthly / weekly income vs expense series	analyst+
GET	/dashboard/recent-activity	Last N posted records for activity feed	viewer+
GET	/dashboard/export	CSV download of filtered records	analyst+

Categories

Method	Path	Description	Access
GET	/categories	List all categories, optional type filter	all roles
POST	/categories	Create custom category	admin
DELETE	/categories/{id}	Delete — blocked if records reference it	admin

Roles & Permissions

Method	Path	Description	Access
GET	/roles	List roles with their permissions	admin
POST	/roles/{role_id}/permissions	Assign a permission to a role	admin
DELETE	/roles/{role_id}/permissions/{perm_id}	Revoke a permission from a role	admin

6. Validation & Error Handling

6.1 Two-Layer Validation

- Pydantic v2 schemas — field validators on every request schema reject invalid values (negative amounts, future dates, bad currency codes, blank strings) before reaching the service. FastAPI returns 422 with field-level error detail automatically.
- Service layer guards — business rule violations that cannot be expressed as field validators (category type mismatch, voided record immutability, last-admin protection) raise typed custom exceptions caught by registered handlers.

6.2 HTTP Status Code Conventions

Code	Meaning	When used in this API
200	OK	Successful GET or PATCH
201	Created	Successful POST — resource created, full object returned
204	No Content	Successful DELETE — no body
401	Unauthorized	Missing, expired, or invalid JWT token — or inactive user account
403	Forbidden	Valid token but permission denied — or forbidden state operation (e.g. delete last admin)
404	Not Found	Resource ID does not exist
409	Conflict	Duplicate email, category in use, update on void record, remove last admin
422	Unprocessable Entity	Pydantic validation failed — field-level errors in response body

7. Project Structure

```
fastapi_app/
├── app/
│   ├── api/
│   │   ├── dependencies.py      # extract_token, get_current_user, require_permission
│   │   ├── error_handlers.py    # maps domain exceptions to HTTP responses
│   │   └── v1/ endpoints/       # auth, users, records, categories, dashboard, roles
│   ├── core/
│   │   ├── config.py            # Pydantic BaseSettings, reads .env
│   │   ├── jwt.py               # create/decode access + refresh tokens
│   │   ├── audit.py             # write_audit() helper called after every mutation
│   │   └── exceptions.py        # NotFoundError, ConflictError, ForbiddenOperationError
│   ├── db/
│   │   ├── session.py           # async engine + get_db dependency
│   │   ├── seed.py              # idempotent role + permission seeder
│   │   └── migrations/          # Alembic migration versions
│   ├── models/
│   │   ├── __init__.py          # imports all models → registers with Base.metadata
│   │   ├── base.py              # Base, TimestampMixin, generate_uuid
│   │   ├── user.py / role.py / permission.py
│   │   ├── user_role.py         # plain Table – no mapped class
│   │   ├── role_permission.py   # plain Table – no mapped class
│   │   └── category.py / financial_record.py / audit_log.py
│   ├── repositories/            # BaseRepository[T] + entity-specific repos
│   ├── schemas/                 # Pydantic v2 request + response schemas
│   ├── services/                # business logic + orchestration
│   └── utils/                   # pagination, date helpers, CSV export
├── tests/ unit/ + integration/
├── main.py                      # app factory + lifespan context manager
├── alembic.ini
├── .env.example                 # committed – documents required variables
├── .env                         # gitignored – real secrets
└── pyproject.toml
```

8. Running the Project

8.1 Setup

```
# 1. Clone and enter the project
git clone <repo-url> && cd fastapi_app
```

```
# 2. Create and activate virtual environment
python -m venv .venv
.venv\Scripts\activate      # Windows
source .venv/bin/activate   # macOS / Linux

# 3. Install dependencies
pip install -r requirements.txt

# 4. Configure environment
cp .env.example .env
# Set SECRET_KEY and REFRESH_SECRET_KEY:
# python -c "import secrets; print(secrets.token_hex(32))"

# 5. Start the server
uvicorn main:app --reload

# The server automatically:
# - Creates finance.db
# - Creates all 8 tables
# - Seeds 17 permissions + 3 roles
# - Serves API at http://127.0.0.1:8000/api/v1
# - Serves docs at http://127.0.0.1:8000/api/docs
```

8.2 Required Environment Variables

Variable	Example	Required
SECRET_KEY	openssl rand -hex 32	Yes — set before starting
REFRESH_SECRET_KEY	different random hex string	Yes — must differ from SECRET_KEY
DATABASE_URL	sqlite+aiosqlite:///./finance.db	No — default is SQLite
ALGORITHM	HS256	No — default HS256
ACCESS_TOKEN_EXPIRE_MINUTES	60	No — default 60
REFRESH_TOKEN_EXPIRE_DAYS	30	No — default 30

✓
OK

The application is fully self-contained. A reviewer can clone the repository, copy `.env.example` to `.env`, set the two secret key values, and run `uvicorn main:app` — the database file, all tables, and all seed data are created automatically on first startup. No separate database installation, migration commands, or seed scripts are required.